

BÀI LUẬN NHÓM

Môn: Thiết kế và Xây dựng Phần mềm

Định hướng chủ đề chung: Tác động của AI

Sĩ số: 39 học viên

Số nhóm: 10 nhóm (9 nhóm 4 người + 1 nhóm 3 người)

Hình thức: Bài luận nhóm + thuyết trình

PHẦN A. QUY ĐỊNH CHUNG

A.1. Yêu cầu về sản phẩm

Mỗi nhóm nộp 02 sản phẩm:

- Bài luận:** 12–18 trang, (font 13), bắt buộc ghi rõ phụ lục bằng chứng tài liệu tham khảo
- Slide thuyết trình:** tối đa 15 trang, trình bày 15 phút + 5 phút hỏi đáp

A.2. Cấu trúc bắt buộc của bài luận

Mục	Nội dung
1. Đặt vấn đề	Bối cảnh, vì sao vấn đề đáng quan tâm với ngành phần mềm Việt Nam
2. Câu hỏi nghiên cứu	2–3 câu hỏi cụ thể, trả lời được
3. Tổng quan tài liệu	Tối thiểu 8 nguồn, trong đó ít nhất 3 nguồn học thuật hoặc báo cáo khảo sát quy mô lớn
4. Phương pháp	Nêu rõ cách thu thập và xử lý dữ liệu
5. Kết quả và phân tích	Số liệu, bảng, biểu đồ do nhóm tự tạo
6. Bàn luận và khuyến nghị	Ý nghĩa với thực tiễn, giới hạn của nghiên cứu
7. Phân công công việc	Bảng ghi rõ đóng góp từng thành viên

A.3. Nguyên tắc bắt buộc về sử dụng AI

Học viên **được phép** dùng công cụ AI trong quá trình làm bài, với ba điều kiện:

- Ghi rõ trong phụ lục: đã dùng công cụ nào, cho phần việc gì
- Mọi con số, trích dẫn, khẳng định thực nghiệm phải truy vết được về nguồn gốc
- Nhóm chịu trách nhiệm hoàn toàn về tính chính xác của nội dung, kể cả phần do AI sinh ra

Vì phạm nguyên tắc truy vết dữ liệu sẽ bị trừ tối thiểu 30% tổng điểm.

A.4. Thang điểm

Tiêu chí	Điểm
Chất lượng nội dung học thuật	40
Bảng chứng thực nghiệm do nhóm tự thu thập	25
Tính ứng dụng vào bối cảnh Việt Nam	15
Thuyết trình và trả lời câu hỏi	15
Chất lượng trình bày, trích dẫn đúng chuẩn	5

PHẦN B. CÁC CHỦ ĐỀ

Chủ đề 1. Khoảng cách giữa điểm benchmark và năng lực thực tế của AI coding agent

Câu hỏi trung tâm: Vì sao một mô hình đạt điểm rất cao trên SWE-bench Verified lại thất bại trên các bài toán phát triển tính năng đầu-cuối?

Bối cảnh cần khảo cứu:

- Điểm SWE-bench Verified tăng từ 1,96% (10/2023) lên 78,4% (04/2026)
- FeatureBench: một mô hình đạt 74,4% trên SWE-bench chỉ giải được 11,0% các bài toán ở mức tính năng, trải nhiều commit và nhiều pull request
- Vấn đề rò rỉ dữ liệu huấn luyện và ô nhiễm benchmark

Việc nhóm phải làm:

- Lập bảng so sánh tối thiểu 4 benchmark: phạm vi đo, cách chấm, điểm yếu phương pháp luận
- Tự thiết kế 5 bài toán nhỏ từ một repo mã nguồn mở, chạy thử với ít nhất 2 công cụ AI, ghi lại tỷ lệ thành công
- Phân tích: đặc điểm nào của bài toán khiến agent thất bại

Sản phẩm đặc thù: Bảng kết quả thực nghiệm 5 bài toán $\times$ 2 công cụ, kèm nhật ký thao tác

Chủ đề 2. Nghịch lý năng suất: AI có thực sự làm lập trình viên nhanh hơn?

Câu hỏi trung tâm: Vì sao cảm nhận chủ quan về năng suất lại lệch xa kết quả đo khách quan?

Bối cảnh cần khảo cứu:

- Nghiên cứu RCT của METR: 16 lập trình viên kinh nghiệm, 246 nhiệm vụ trên chính repo của họ. Họ dự đoán nhanh hơn 24%, sau khi làm xong vẫn tin mình nhanh hơn 20%, nhưng thực tế **chậm hơn 19%**
- METR sau đó phải hủy thí nghiệm tiếp theo vì tín hiệu dữ liệu không đáng tin do sai lệch chọn mẫu
- Các nghiên cứu có kiểm soát khác lại ghi nhận tiết kiệm 13,6%–55,8% thời gian
- Thời gian dịch chuyển: bớt gõ code và tìm kiếm, tăng thời gian viết prompt, chờ đợi và kiểm tra kết quả

Việc nhóm phải làm:

- Thiết kế một thí nghiệm nhỏ trong nhóm: mỗi thành viên làm 4 nhiệm vụ, 2 có AI và 2 không, đo thời gian thực tế và thời gian tự ước lượng
- Đối chiếu độ lệch giữa cảm nhận và thực đo
- Giải thích các yếu tố gây nhiễu và giới hạn của thí nghiệm quy mô nhỏ

Sản phẩm đặc thù: Bộ dữ liệu thời gian đo được và biểu đồ độ lệch cảm nhận – thực tế

Chủ đề 3. An toàn bảo mật của mã nguồn do AI sinh ra

Câu hỏi trung tâm: Mức độ và dạng lỗ hổng phổ biến trong mã do AI sinh, và quy trình nào ngăn được chúng?

Bối cảnh cần khảo cứu:

- Veracode: thử nghiệm trên 100+ mô hình, 45% mẫu mã sinh ra chứa lỗ hổng thuộc OWASP Top 10; riêng Java tỷ lệ trượt tới 72%; 86% mẫu không chống được XSS; 88% dính log injection
- Nghiên cứu kiểm chứng hình thức trên 3.500 mẫu từ 7 mô hình: 55,8% chứa ít nhất một lỗ hổng, và **các công cụ SAST công nghiệp bỏ sót 97,8% số lỗi** đã được chứng minh
- Số CVE quy được cho công cụ AI trong năm 2026: tháng 1 là 6, tháng 2 là 15, tháng 3 là 35
- Chỉ dẫn bảo mật tường minh trong prompt chỉ giảm tỷ lệ lỗi khoảng 4 điểm phần trăm

Việc nhóm phải làm:

- Chọn 5 loại CWE, viết prompt yêu cầu AI sinh code cho từng loại, quét bằng công cụ phân tích tĩnh miễn phí
- Lập lại thí nghiệm với prompt có kèm chỉ dẫn bảo mật, so sánh kết quả
- Đề xuất một quy trình công kiểm soát bảo mật cho mã do AI sinh

Sản phẩm đặc thù: Bảng kết quả quét, mã nguồn mẫu, đề xuất quy trình dạng sơ đồ

Chủ đề 4. Nguy cơ kỹ thuật tiềm ẩn trong mã do AI sinh

Câu hỏi trung tâm: Tăng tốc độ tạo code hôm nay có làm tăng chi phí bảo trì ngày mai không?

Bối cảnh cần khảo cứu:

- Nghiên cứu tại MSR 2026 phân tích 302.579 commit được xác nhận do AI tạo trên 6.299 kho mã nguồn, gồm 5 công cụ phổ biến
- Tổng quan hệ thống 34 nghiên cứu giai đoạn 2024–2026: 58% ghi nhận suy giảm chất lượng ở ít nhất một chỉ số; trùng lặp mã tăng ở 52% nghiên cứu; độ phức tạp chu trình tăng ở 48%; mật độ lỗi tăng trung vị 23% ở các module nhiều mã AI
- Nợ kỹ thuật tăng trung bình 1,8 người-ngày cho mỗi 1.000 dòng mã do AI sinh
- Ghi nhận tại một số doanh nghiệp lớn: lập trình viên dùng AI tạo commit nhanh gấp 3–4 lần nhưng phát sinh cảnh báo bảo mật gấp 10 lần

Việc nhóm phải làm:

- Chọn một dự án mã nguồn mở có dấu vết commit do AI tạo, đo các chỉ số chất lượng theo thời gian
- So sánh module nhiều mã AI với module ít mã AI trên cùng dự án
- Đề xuất bộ chỉ số cảnh báo sớm nợ kỹ thuật cho đội dùng AI

Chủ đề 5. Specification-driven development: đặc tả trở lại làm trung tâm

Câu hỏi trung tâm: Khi AI viết được phần lớn code, đặc tả thay đổi vai trò như thế nào?

Bối cảnh cần khảo cứu:

- GitHub Spec Kit và luồng bốn bước Spec → Plan → Tasks → Implement, với khái niệm "constitution" ghi các nguyên tắc bất di bất dịch của tổ chức
- Sự khác biệt giữa prompt (ngữ cảnh tạm thời) và specification (tri thức bền vững của dự án)
- Liên hệ ngược về use case, luồng sự kiện và tiêu chí nghiệm thu đã học

Việc nhóm phải làm:

- Chọn một chức năng cụ thể, thực hiện hai cách: (a) prompt trực tiếp nhiều vòng, (b) viết đặc tả trước rồi mới giao cho AI
- So sánh kết quả trên bốn tiêu chí: đúng yêu cầu, độ bao phủ ngoại lệ, khả năng đọc hiểu, số vòng sửa
- Đánh giá: đặc tả kiểu use case cổ điển và đặc tả kiểu Spec Kit khác nhau ở đâu

Sản phẩm đặc thù: Hai bộ hiện thực song song và bảng so sánh định lượng

Chủ đề 6. Kiến trúc phần mềm trong kỷ nguyên agent: A-SDLC

Câu hỏi trung tâm: Vòng đời phát triển phần mềm thay đổi ra sao khi agent tham gia thực thi?

Bối cảnh cần khảo cứu:

- Đề xuất kiến trúc tham chiếu sáu tầng cho hệ thống kỹ nghệ phần mềm có agent
- So sánh SDLC truyền thống với Agentic SDLC (A-SDLC)
- Năm bài toán mở được nêu: đánh giá, quản trị, nợ kỹ thuật, tái phân bổ kỹ năng, kinh tế học của sự chú ý
- Hiện tượng xói mòn kiến trúc khi agent tối ưu cục bộ mà không thấy toàn cảnh

Việc nhóm phải làm:

- Vẽ lại sơ đồ A-SDLC bằng ngôn ngữ của nhóm, đối chiếu từng pha với SDLC cổ điển
- Chỉ ra pha nào chịu thay đổi lớn nhất và pha nào gần như không đổi
- Thiết kế một bộ quy tắc kiến trúc mà agent phải tuân thủ, dạng file cấu hình đọc được bởi máy

Chủ đề 7. Kiểm thử và kiểm chứng khi AI viết cả code lẫn test

Câu hỏi trung tâm: Làm sao phá vỡ vòng lặp "AI hiểu sai yêu cầu → viết code sai → viết test chứng minh code sai là đúng"?

Bối cảnh cần khảo cứu:

- Hiện tượng ảo tưởng kiểm thử: bộ test đầy đủ nhưng cùng dựa trên một giả định sai
- Vai trò của kiểm chứng độc lập, property-based testing, mutation testing
- Tiêu chí nghiệm thu do con người viết như một lớp chốt chặn

Việc nhóm phải làm:

- Chuẩn bị một yêu cầu cố tình mơ hồ, để AI tự sinh cả hiện thực lẫn test, xem nó có phát hiện được sự mơ hồ không
- Áp dụng mutation testing hoặc property-based testing lên chính bộ test đó, đo tỷ lệ phát hiện lỗi
- Đề xuất mô hình phân tách trách nhiệm: ai viết đặc tả, ai viết test, ai viết code

Sản phẩm đặc thù: Báo cáo mutation score và phân tích các test "giả mạnh"

Chủ đề 8. Code review khi khối lượng thay đổi vượt năng lực đọc của con người

Câu hỏi trung tâm: Nút thắt đã chuyển từ viết code sang review — làm gì với nó?

Bối cảnh cần khảo cứu:

- Pull request 200 dòng và pull request 5.000 dòng khác nhau ra sao về chất lượng review thực tế
- Nghiên cứu ghi nhận: người review thường không bắt được nợ kỹ thuật do agent tạo ra, và gần một phần tư khuyết tật tồn tại vô thời hạn
- AI hỗ trợ review: tóm tắt pull request, phát hiện code smell, gợi ý refactor — và giới hạn của chúng

Việc nhóm phải làm:

- Khảo sát tối thiểu 15 kỹ sư đang đi làm về thời gian review trung bình và kích thước PR
- Thực nghiệm: cho một nhóm review PR lớn do AI tạo, đo tỷ lệ phát hiện lỗi đã cấy sẵn
- Đề xuất chính sách giới hạn kích thước thay đổi và quy trình review phân tầng

Sản phẩm đặc thù: Kết quả khảo sát và tỷ lệ phát hiện lỗi cấy sẵn

Chủ đề 9. Dịch chuyển kỹ năng và chương trình đào tạo kỹ sư phần mềm

Câu hỏi trung tâm: Nội dung nào trong chương trình đào tạo nên giảm, nên giữ, nên tăng?

Bối cảnh cần khảo cứu:

- Khảo sát ghi nhận khoảng 49% công việc được lấy mẫu có AI tham gia ít nhất một phần tư số nhiệm vụ
- Nguy cơ automation bias: người học chưa vững kiến trúc rất khó nhận ra thiết kế do AI đề xuất có vấn đề
- Các năng lực được cho là tăng giá trị: đặt bài toán, kỹ nghệ yêu cầu, kiến trúc, kiểm chứng, đọc code, tri thức miền, phán đoán kỹ thuật

Việc nhóm phải làm:

- Phân tích chương trình đào tạo ngành CNTT của tối thiểu 2 trường đại học Việt Nam
- Phỏng vấn tối thiểu 5 người: kỹ sư đang đi làm, quản lý kỹ thuật hoặc giảng viên
- Đề xuất một đề cương môn học cập nhật, nêu rõ học phần nào tăng, giảm, giữ nguyên và vì sao

Sản phẩm đặc thù: Bảng đối chiếu chương trình và biên bản phỏng vấn

Chủ đề 10. Quản trị và trách nhiệm pháp lý khi AI tham gia phát triển phần mềm

Câu hỏi trung tâm: Tổ chức cần cơ chế kiểm soát nào để dùng AI mà vẫn chịu trách nhiệm được về sản phẩm?

Bối cảnh cần khảo cứu:

- Ba lớp kiểm soát: kiểm soát ngữ cảnh, kiểm soát thực thi, kiểm chứng đầu ra
- Câu hỏi thực tế: AI có được đọc dữ liệu vận hành không? Có được đẩy code lên môi trường thật không? Ai chịu trách nhiệm nếu bản sửa gây mất dữ liệu?
- Vấn đề bản quyền và giấy phép mã nguồn do AI sinh
- Nghĩa vụ bảo vệ dữ liệu cá nhân theo quy định hiện hành của Việt Nam

Việc nhóm phải làm:

- Soạn một bộ chính sách sử dụng AI trong phát triển phần mềm cho một tổ chức giả định, dài 4–6 trang
- Đối chiếu với ít nhất 2 khung quản trị đã công bố
- Chỉ rõ điểm nào của chính sách là bắt buộc, điểm nào là khuyến nghị, và cách kiểm tra tuân thủ

Sản phẩm đặc thù: Bản dự thảo chính sách hoàn chỉnh, có thể đem áp dụng thật